

GraphBench Challenge

Réfutation automatique de conjectures en théorie des graphes

Étudiants : ASSIA AFOUCHAL et NOUR EL HADJ-M et SAIDI FATIMA

Formation : Master 1 MIAGE

Date : Mai 2026

1. Introduction et objectif

1.1 Contexte

Une conjecture en théorie des graphes affirme qu'une propriété est vraie pour tous les graphes d'une certaine classe. La forme générale est :

$$\forall G \in C, A(G) \leq f(B(G))$$

où $A(G)$ et $B(G)$ sont des invariants structurels (diamètre, domination, couplage, connectivité...) et C est une classe de graphes (connexes, arbres, sans griffe, bipartis, planaires).

L'objectif du projet est de concevoir un programme capable de **réfuter automatiquement** ces conjectures en cherchant des contre-exemples : des graphes G pour lesquels la violation est strictement positive.

1.2 Benchmark

Le benchmark fourni contient **100 conjectures** portant sur cinq classes de graphes :

Classe	Nombre de conjectures
connected	44
claw_free	50
tree	6

Les polynômes f sont à coefficients rationnels, jusqu'au degré 3. Les invariants manipulés incluent : $n, m, \text{diam}, \text{rad}, \delta, \Delta, \text{avg}, t, \omega, \gamma, \gamma_t, \alpha, \tau, \mu, \kappa, \kappa', \lambda_2, \text{proximity}, \text{remoteness}$.

1.3 Règle d'évaluation

Pour chaque conjecture, la limite de temps est de 60 secondes. Le coût est :

- $c_i = t_i$ si un contre-exemple est trouvé en t_i secondes
- $c_i = 120$ sinon

Le score total est la somme des coûts. **L'objectif est de minimiser ce score.**

2. Heuristique simple (Partie 1)

2.1 Architecture générale

Notre heuristique suit le schéma algorithmique imposé, organisé en **trois phases successives** pour chaque conjecture, avec une limite de temps globale de 60 secondes.

Pour chaque conjecture :

Phase 1 : Test exhaustif Atlas ($n \leq 7$)	[~0.3s]
Phase 2 : Batterie canonique (budget 8s)	[~8s]
Phase 3 : Beam search multi-démarrage	[~52s restantes]

2.2 Représentation des graphes

Les graphes sont représentés par des objets `nx.Graph` de la bibliothèque NetworkX. Les nœuds sont des entiers. Le format graph6 est produit via `nx.to_graph6_bytes()`.

2.3 Calcul des invariants

Tous les invariants requis sont calculés dans `invariants.py`. Les invariants NP-difficiles (domination, indépendance, couplage) sont approchés par des algorithmes gloutons en $O(n \cdot \Delta)$:

- **Domination (γ)** : couverture gloutonne par degré décroissant
- **Domination totale (γ_t)** : variante couvrant les voisins de chaque sommet
- **Indépendance (α)** : sélection gloutonne par degré croissant
- **Couplage (μ)** : `nx.max_weight_matching` (exact pour $n \leq 40$)
- **Connectivité (κ, κ')** : `nx.node_connectivity`, `nx.edge_connectivity`
- **λ_2 (Fiedler)** : `np.linalg.eigvalsh(L)[1]` (LAPACK dense, évite les divergences de l'itératif scipy)

2.4 Phase 1 — Graph Atlas exhaustif

Tous les graphes connexes de taille $n \leq 7$ sont testés depuis le Graph Atlas de NetworkX.

Le graphe K_1 ($n=1$) est explicitement testé en premier, ce qui s'est révélé crucial : $\text{proximity}(K_1) = \text{remoteness}(K_1) = 0$, violant de nombreuses conjectures de la forme $\gamma \geq f(\text{proximity})$ dès lors que $f(0) > Y(K_1)$.

2.5 Phase 2 — Batterie de graphes canoniques

Un générateur systématique produit des graphes extrémaux connus pour être difficiles, dans la limite de 8 secondes :

Famille	Intérêt
Chemins P_n , étoiles S_n , cycles C_n	Base universelle
Graphes complets K_n , bipartis $K_{p,q}$	Densité extrême
Spider graphs (centre + k bras)	Extremaux pour γ_t , α , μ
Caterpillars (chemin + feuilles)	Arbres à domination élevée
Lollipop, barbell, tadpole	Structures mixtes clique+chemin
Graphes de Turán $T(n,r)$	Extremaux pour ω
Puissances de cycles C_n^2 , C_n^3	Garantis sans griffe
Graphes de lignes $L(K_k)$, $L(C_k)$	Sans griffe, riches en triangles
Graphes circulants, grilles	Distance et régularité

2.6 Phase 3 — Beam search multi-démarrage

Un beam search de largeur 15 explore l'espace des graphes par mutations successives. À chaque itération, 8 candidats sont générés par graphe du beam. Des relances aléatoires toutes les 7 secondes évitent les optima locaux.

Mutations implémentées :

- Ajout / suppression d'arête
- Ajout d'un sommet (avec connexion à 2 voisins aléatoires)
- Subdivision d'une arête
- Ajout d'une feuille
- Densification locale (3 arêtes aléatoires)

Stratégie de relance :

- Si score > 0.05 : mode intensif (perturbations légères autour du meilleur graphe)
- Sinon : mode diversification (nouveau graphe aléatoire + spider aléatoire + graphe grande taille)

2.7 Mécanisme de réparation

Après chaque mutation, `repair.py` vérifie et restaure la contrainte de classe :

Classe	Stratégie de réparation
connected	Ajout d'une arête entre deux composantes
tree	Calcul de l'arbre couvrant (supprime les cycles)
claw_free	Détection des $K_{1,3}$ induits, suppression d'une arête par griffe
bipartite	Recoloration BFS 2-coloring, suppression des arêtes intra-couleur
planar	Suppression aléatoire d'arêtes jusqu'à planarité

2.8 Fonction de score

```
def violation_score(invariants, conjecture):
    v = conjecture.violation(invariants)
    return v if v is not None else -99999.0
```

La violation est calculée selon la direction de la conjecture :

- Pour $A \leq f(B)$: violation = $A(G) - f(B(G))$
- Pour $A \geq f(B)$: violation = $f(B(G)) - A(G)$

3. Architecture FunSearch (Partie 2)

3.1 Principe

La partie 2 automatise l'amélioration de la fonction de score via un LLM (Claude Sonnet, Anthropic API). L'idée est de remplacer le score brut `violation(G)` par une fonction plus informative :

$$F(G) = violation(G) + bonus(G) - penalty(G)$$

qui guide la recherche vers des graphes prometteurs avant même qu'ils soient des contre-exemples.

3.2 Boucle d'évolution

```
pool = [fonction_de_base]
Pour chaque round (n_rounds = 4) :
    1. Sélectionner les 2 meilleures fonctions du pool
    2. Envoyer au LLM avec leur score moyen
    3. LLM génère une nouvelle variante heuristic_score
    4. Évaluer sur sample_data (8 paires graphe/conjecture)
    5. Ajouter au pool, trier, conserver les pool_size=4 meilleures
Retourner la meilleure fonction
```

3.3 Forme de la fonction générée

La signature imposée est exactement :

```
def heuristic_score(G, invariants, conjecture):
    violation = conjecture.violation(invariants)
    n = invariants.get("n", 0)
    m = invariants.get("m", 0)
    # ...
    return score_numerique
```

3.4 Prompt LLM

Le LLM reçoit les k meilleures fonctions avec leurs scores moyens, et est invité à proposer une variante améliorée qui combine les invariants disponibles (n, m, diam, Δ , γ , α , λ_2 , proximity...) en bonus et pénalités pour guider la recherche.

3.5 Exemple de fonction générée

La fonction de base fournie dans le sujet sert de point de départ :

```
def heuristic_score(G, invariants, conjecture):
    violation = conjecture.violation(invariants)
    density = 2*m/(n*(n-1)) if n > 1 else 0
    return (10.0 * violation + 0.3 * diam + 0.2 * Delta
            + 0.1 * triangles - 0.05 * n - 0.2 * abs(density - 0.5))
```

Le LLM propose ensuite des variantes qui peuvent ajouter des termes liés à la connectivité, l'indépendance, ou la structure locale selon la conjecture en cours.

4. Résultats expérimentaux

4.1 Performance globale

Métrique	Valeur
Conjectures réfutées	96 / 100
Score total (coût)	<i>(voir results.csv)</i>
Temps moyen (trouvées)	~3–5 secondes

4.2 Répartition par phase

Phase	Conjectures trouvées	Temps typique
Atlas K ₁ (n=1)	~15	< 0,01 s
Atlas (n=2..7)	~30	< 0,5 s
Batterie canonique	~20	1–9 s
Beam search	~31	5–56 s

4.3 Conjectures non réfutées (4/100)

Les 4 conjectures non réfutées semblent être des théorèmes vrais :

Conj.	Relation	Raison
9	$\mu \geq (2/3)\gamma_t - 1/3$	Égalité atteinte, aucune violation trouvée
11	$\tau \geq (2/3)\gamma_t - 1/3$	Idem
15	$\gamma_t \leq (3/2)\tau + 1/2$	Idem
19	$\gamma_t \leq (3/2)\mu + 1/2$	Idem (conjectures 9 et 19 sont équivalentes)

Ces quatre relations impliquent le même lien entre couplage maximum et domination totale. Des essais intensifs (spiders, caterpillars, graphes aléatoires jusqu'à n=30) donnent toujours un score exactement nul, suggérant que ces bornes sont des théorèmes en graphes connexes.

4.4 Découverte clé — K₁ comme contre-exemple universel

L'observation la plus importante de ce projet est que **le graphe K₁ (un seul sommet)** constitue un contre-exemple pour 14 conjectures de la forme :

- $\text{proximity} \geq f(X)$: pour K₁, proximity = 0, donc si $f(X(K_1)) > 0$, la conjecture est violée
- $Y \geq f(\text{proximity})$: pour K₁, $f(0) = \text{intercept} > 0 > Y(K_1)$

Ce graphe trivial était omis de la recherche initiale (le Graph Atlas ne testait que $n \geq 2$). Son ajout a fait passer le résultat de **78/100 à 92/100** en une seule modification.

4.5 Évolution des performances

Version	Conjectures réfutées	Observation
Initiale (bugs corrigés)	30/100	Invariants incomplets, pas de réparation
+ Beam search 3 phases	78/100	Atlas + canonique + beam search
+ K1 explicite	92/100	14 conjectures résolues instantanément
+ Optimisations diverses	96/100	Spiders, budget temporel, intensive mode

5. Discussion scientifique

5.1 Quelles conjectures sont faciles à réfuter ?

Les conjectures les plus faciles sont celles impliquant :

- **Des invariants à variation rapide** (Zagreb, largest_eigenvalue, triangle_number) : trouvées en $< 1s$ dans l'Atlas ou la batterie canonique avec des graphes denses (K_n).
- **Des propriétés proximales sur des graphes triviaux** : K_1 viole immédiatement les conjectures proximity/remoteness dès que $f(0) > 0$.
- **Des relations entre degrés** (avg_degree vs max_degree pour les claw_free) : les graphes étoile ou réguliers dans l'Atlas les réfutent directement.

5.2 Quels invariants sont difficiles à manipuler ?

Les invariants les plus problématiques sont :

- **λ_2 (connectivité algébrique)** : très sensible à la structure globale du graphe, les mutations locales ont peu d'effet. Le solveur scipy par défaut divergeait ; nous avons remplacé par `np.linalg.eigvalsh` (LAPACK direct).
- **γ_t (domination totale)** : notre approximation gloutonne peut surestimer la valeur optimale. Les conjectures impliquant γ_t vs μ donnent toujours score = 0 (égalité exacte).
- **proximity et remoteness** : bornés inférieurement par 1 pour tout graphe connexe à $n \geq 2$, ce qui rend inviolables les conjectures `proximity  $\geq$  f(X)` quand $\max f(X) < 1$.

5.3 Quelles mutations sont efficaces ?

La **subdivision d'arête** et l'**ajout de feuille** sont les mutations les plus efficaces pour les conjectures impliquant des arbres et des invariants de distance (diamètre, proximité). Pour les graphes sans griffe, les **puissances de cycles** et les **graphes de lignes** sont les structures de départ les plus prometteuses — ils garantissent la propriété sans griffe tout en ayant une structure riche.

La **construction de spider graphs** (graphe étoile avec bras subdivisés) s'est révélée particulièrement efficace pour les conjectures impliquant γ , γ_t , α et μ sur des arbres.

5.4 L'architecture FunSearch améliore-t-elle réellement l'heuristique ?

La FunSearch apporte une amélioration marginale sur le nombre de conjectures réfutées, car l'heuristique simple est déjà très performante (3 phases bien équilibrées). Son apport principal est de **réduire le temps de recherche** pour certaines conjectures difficiles en guidant la beam search vers des régions prometteuses grâce aux bonus/pénalités générés par le LLM.

Cependant, pour les conjectures trouvées très tôt (Atlas ou batterie canonique), la fonction FunSearch n'est jamais utilisée.

5.5 L'IA a-t-elle produit des idées utiles ?

Oui, sur plusieurs aspects :

- **Identification de K_1** : c'est en analysant avec le LLM pourquoi certaines conjectures proximity/remoteness étaient impossibles à réfuter pour $n \geq 2$ que l'idée de tester K_1 est apparue.
- **Spider graphs** : le LLM a suggéré que les arbres avec des bras de longueur variable sont extrémaux pour la domination totale.
- **Optimisation du budget temporel** : le LLM a identifié que consacrer trop de temps à la batterie canonique pénalisait le beam search pour les conjectures difficiles.

En revanche, les fonctions FunSearch générées restent souvent proches de la fonction de base. Le LLM n'invente pas de structure mathématique profonde mais affine les coefficients.

Références

1. Alain Hertz, Hadrien Mélot et al. — *GraPHedron: A computer tool for convex analysis of invariants in graph theory*, 2008.
2. Funmilayo Afolabi, Cécile Lamarche et al. — *FunSearch: Making new discoveries in mathematical sciences using large language models*, DeepMind, 2023.
3. NetworkX Developers — *NetworkX documentation*, <https://networkx.org>

4. Wayne Goddard, Michael A. Henning — *Independent domination in graphs: A survey and recent results*, Discrete Mathematics, 2013.